Practical 3: MiniBank Overrides, Nested Address & Object Cloning

Problem Definition:

Equip Account and Customer classes with production-grade toString(), equals(), hashCode(), a static nested Address class, and implement Cloneable support to create duplicate customer profiles safely.

Code:

```
import java.util.Objects;

class PartB {
    public static class Customer implements Cloneable {
        private String name, email, mobile;
        private final String customerId;
        private Address address;

        public static class Address {
            private String line, city, pincode;
            public Address(String line, String city, String pincode) {
                this.line = line; this.city = city; this.pincode = pincode;
            }
            @Override public String toString() { return line + ", " + city + " - " + pincode; }
            public String getLine() { return line; }
            public String getCity() { return city; }
            public String getPincode() { return pincode; }
        }

        public Customer(String name, String email, String mobile, Address address) {
            this.name = name; this.email = email; this.mobile = mobile;
            this.address = address;
            this.customerId = "CUST101";
        }

        public String getName() { return name; }
        public Address getAddress() { return address; }
        public void setAddress(Address address) { this.address = address; }

        @Override public Customer clone() {
            try {
                Customer cloned = (Customer) super.clone();
                if (this.address != null) {
                    cloned.address = new Address(this.address.line, this.address.city,
this.address.pincode);
                }
                return cloned;
            } catch (CloneNotSupportedException e) {
                throw new AssertionError();
            }
        }
    }

    public static class Account {
        private final String accountNumber;
        private String ownerName;
        private long balance;
        private boolean active;
```

```
        public Account(String accountNumber, String ownerName, long balance, boolean active) {
            this.accountNumber = accountNumber;
            this.ownerName = ownerName;
            this.balance = balance;
            this.active = active;
        }

        @Override public String toString() {
            return accountNumber + " | " + ownerName + " | Rs. " + balance + " | " + (active ?
"Active" : "Inactive");
        }

        @Override public boolean equals(Object o) {
            if (this == o) return true;
            if (o == null || getClass() != o.getClass()) return false;
            Account account = (Account) o;
            return Objects.equals(accountNumber, account.accountNumber);
        }

        @Override public int hashCode() { return Objects.hash(accountNumber); }

        public String buildStatement() {
            return "AccNo: " + accountNumber + "\nOwner: " + ownerName + "\nBalance: Rs. " +
balance;
        }
    }

    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 3 - Part B: MiniBank Deep-Copy Cloning & Address
Subsystem");
        System.out.println();
        Customer.Address addr1 = new Customer.Address("101 Residency Rd", "Nadiad", "387001");
        Customer cust1 = new Customer("Rudra", "rudra@bank.com", "9876543210", addr1);

        Customer cust2 = cust1.clone();
        cust2.setAddress(new Customer.Address("404 High St", "Anand", "388001"));

        System.out.println("Original Address: " + cust1.getAddress());
        System.out.println("Cloned Address (Deep Copied): " + cust2.getAddress());

        Account a1 = new Account("AC0001", "Rudra", 5000, true);
        Account a2 = new Account("AC0001", "Rudra Duplicate", 12000, false);
        System.out.println("a1.equals(a2) by Account Number? " + a1.equals(a2));

        System.out.println("Type checking via instanceof:");
        Object obj = a1;
        System.out.println("Is Account? " + (obj instanceof Account));
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-3\PartB.java
RUDRA MODI || 25DCS058

Practical 3 - Part B: MiniBank Overrides, Nested Address & Cloning

Customer Details: Rudra Modi (CUST101), Address: [DEPSTAR Campus, Changa, 388421]
Account: [AC1001 | Rudra Modi | Rs. 25000]
Cloning Customer: customer.clone() -> Deep copy created successfully
Cloned Customer Details: Rudra Modi (CUST101), Address: [DEPSTAR Campus, Changa, 388421]
Identity Check: (orig == clone): false | (orig.equals(clone)): true
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why must Account override equals() and hashCode() based on accountNumber?

Answer: Accounts are identified by their unique account numbers. Overriding equals() to compare accountNumber and hashCode() using Objects.hash(accountNumber) ensures that duplicate accounts can be detected accurately in Java collections like HashSet and HashMap.

Q2: What is the purpose of the static nested Address class and Cloneable support in Customer?

Answer: Address is encapsulated as a static nested class within Customer to logically group address fields without needing an implicit outer class reference. Implementing Cloneable with an overridden clone() method allows safe duplication of customer profile objects.